

```
In [301... import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sympy as sp
from scipy.optimize import curve_fit
```

Formula of inertia

$$I = m * r(cm)^2$$

The measurements we got by trying to balance our pendulum to find the center of mass is as follows

```
In [306... r_cm = 28.5e-2 ### Meters
d_r_cm = 2e-2

mass = 340.3e-3 ### Kilo Gram
mass_projectile = 66e-3 ### Kilo Gram
d_mass = 0.1e-3
```

Here, we can see that the dominant source of uncertainty is the radius, so we can neglect the other uncertainties and only consider the uncertainty of the radius of the center of mass.

```
In [309... inertia = mass * r_cm ** 2
print(f"Inertia = {inertia:.2g}")
```

Inertia = 0.028

$$\text{Initial Angular Momentum: } L_{\text{initial}} = R_b \cdot m \cdot v$$

$$\text{Final Angular Momentum: } L_{\text{final}} = I \cdot \omega$$

```
In [313... rb = 30e-2 # meters
d_rb = 2e-2
```

To measure the period what we did was we released the pendulum from 10 degrees and count how many seconds it would take for it to travel 10 oscillations

- The pendulum moved 10 oscillations in 10.35 seconds. The uncertainty is about 1 seconds through out the entire osciltion

```
In [316... period = 10 / 10.35  
d_period = 1 / 10
```

Here is the summury of all data points and the angles we got

```
In [319... data = pd.read_csv("data/data.csv")
```

```
In [320... data.head(15)
```

```
Out[320...

|    | Trial        | Angle | d_angle |
|----|--------------|-------|---------|
| 0  | short_range  | 17.0  | 3       |
| 1  | short_range  | 17.0  | 3       |
| 2  | short_range  | 17.5  | 3       |
| 3  | short_range  | 17.5  | 3       |
| 4  | short_range  | 15.0  | 3       |
| 5  | midium_range | 28.0  | 3       |
| 6  | midium_range | 28.0  | 3       |
| 7  | midium_range | 28.0  | 3       |
| 8  | midium_range | 28.0  | 3       |
| 9  | midium_range | 27.5  | 3       |
| 10 | long_range   | 37.0  | 3       |
| 11 | long_range   | 38.0  | 3       |
| 12 | long_range   | 37.0  | 3       |
| 13 | long_range   | 37.5  | 3       |
| 14 | long_range   | 38.0  | 3       |


```

The dominant source of uncertainty is due to inconsistencies in the angle measurement caused by errors during measurement and the friction the pendulum experiences while moving.

To compute the muzzle velocity we need to measure the mass, the distance between the pendulum pivot and where the ball was captured and the moment of inertia

$$gram = 66 \text{ g} \pm 0.1 \text{ g}, \quad R_b = 30 \text{ cm} \pm 1 \text{ cm}$$

Momentum Before = Momentum After

```
In [327... distance = 24.46e-3
spring_x1 = 51.05e-3
spring_x2 = 70.03e-3
spring_x3 = 92.66e-3
uncertainty = 3e-3
```

Now we are going to get the average angle of our five launches to estimate the velocity.

```
In [329... data = data.groupby("Trial")["Angle"].mean()
```

```
In [331... data.head()
```

```
Out[331... Trial
long_range      37.5
midium_range    27.9
short_range     16.8
Name: Angle, dtype: float64
```

On the next block of code we define the velocity function of the projectile before hitting the pendulum

```
In [334... velocity = lambda mass_projectile, mass, rb, r_cm, theta: (1/(mass_projectile * rb))
```

```
In [336... velocities = [velocity(mass_projectile, mass, rb, r_cm, data[row]) for row in data.
data = data.to_frame()
```

```
In [337... data["Velocity"] = velocities
data.head()
```

```
Out[337...      Angle  Velocity
Trial
long_range  37.5  5.265356
midium_range 27.9  3.948943
short_range  16.8  2.392921
```

```
In [339... data["Velocity ** 2"] = data["Velocity"] ** 2
data["distance ** 2"] = (np.array([spring_x1, spring_x2, spring_x3]) - distance) **
data["d_uncertainty"] = 0.1 * np.ones(len(data))
```

```
data = data.sort_values(by="Angle")
data.head()
```

Out[339...

	Angle	Velocity	Velocity ** 2	distance ** 2	d_uncertainty
Trial					
short_range	16.8	2.392921	5.726070	0.004651	0.1
midium_range	27.9	3.948943	15.594149	0.002077	0.1
long_range	37.5	5.265356	27.723974	0.000707	0.1

Estimating the velocity error

In [342...

```
linear_model = lambda x, A, B: (A*x) + B
parameters, cov = curve_fit(linear_model, data["distance ** 2"], data["Velocity ** 2"])
slope = parameters[0]
intercept = parameters[1]
k_value = mass_projectile * slope
data["verr"] = [0.35, 0.51, 0.61]
print(f"Slope = {slope}\nIntercept = {intercept}\nK Value = {k_value}")
```

Slope = -5352.336056267942

Intercept = 29.612746170019435

K Value = -353.25417971368415

Dominant Source of Uncertainty (For Velocity)

$$\Delta v = \sqrt{\left(\frac{\partial v}{\partial t} \Delta t\right)^2 + \left(\frac{\partial v}{\partial M} \Delta M\right)^2 + \left(\frac{\partial v}{\partial \theta} \Delta \theta\right)^2 + \left(\frac{\partial v}{\partial R_m} \Delta R_m\right)^2}$$

Error Propagation (For Velocity)

$$\Delta v = \sqrt{\left(\frac{\partial v}{\partial t} \Delta t\right)^2} = \Delta t \times \frac{\sqrt{\frac{2I}{MR_m^2}} \left(1 - \cos \frac{\theta}{180}\right)}{2tR_m}$$

Result (For Velocity)

$$V_{\text{short}} = 2.242 \pm 0.35 \text{ m/s}$$

$$V_{\text{medium}} = 3.60 \pm 0.51 \text{ m/s}$$

$$V_{\text{long}} = 5.3 \pm 0.61 \text{ m/s}$$

$$\frac{k}{m} = \frac{v^2}{x^2}$$

```
In [347... X = np.linspace(min(data["distance ** 2"]), max(data["distance ** 2"]))
# Plotting
plt.figure(figsize=(10, 6))
plt.errorbar(
    data["distance ** 2"],
    data["Velocity ** 2"],
    yerr=data["verr"],
    fmt="o",
    label="Data points",
    color="blue",
    ecolor="gray",
    capsize=4,
    alpha=0.8,
    markersize=6
)

# Plot the best fit line
plt.plot(
    X,
    linear_model(X, slope, intercept),
    color="red",
    linewidth=2,
    linestyle="--",

    label="K value = 360 +/- 2 (kg/m^2)"
)

# Labels and title
plt.xlabel("Distance² (m²)", fontsize=12)
plt.ylabel("Velocity² (m/s)²", fontsize=12)
plt.title("Visualizing relationship to estimate spring constant", fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6)
# Display the plot
plt.show()
```

